

TRADITIONAL COMPUTER VISION

Core Techniques

- ▶ **Thresholding**

Pixels $\geq$ a value $\rightarrow$ set to maximum

- ▶ Edge Detection

Detect changes in brightness

- ▶ Segmentation

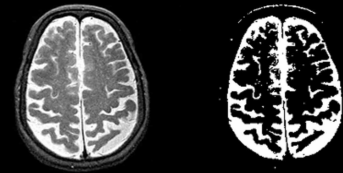
Group regions enclosed by edges

- ▶ Curve Detection

Link adjacent edges into shapes

- ▶ Optical Flow

Estimate motion between frames

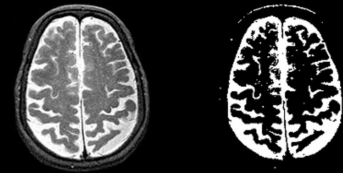


10.4018/978-1-5225-0140-4.ch004

TRADITIONAL COMPUTER VISION

Core Techniques

- ▶ **Thresholding**
Pixels $\geq$ a value $\rightarrow$ set to maximum
- ▶ **Edge Detection**
Detect changes in brightness
- ▶ **Segmentation**
Group regions enclosed by edges
- ▶ **Curve Detection**
Link adjacent edges into shapes
- ▶ **Optical Flow**
Estimate motion between frames

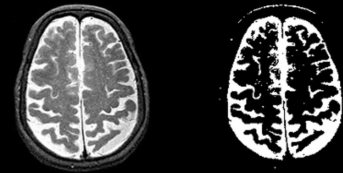


10.4018/978-1-5225-0140-4.ch004

TRADITIONAL COMPUTER VISION

Core Techniques

- ▶ **Thresholding**
Pixels $\geq$ a value $\rightarrow$ set to maximum
- ▶ **Edge Detection**
Detect changes in brightness
- ▶ **Segmentation**
Group regions enclosed by edges
- ▶ **Curve Detection**
Link adjacent edges into shapes
- ▶ **Optical Flow**
Estimate motion between frames

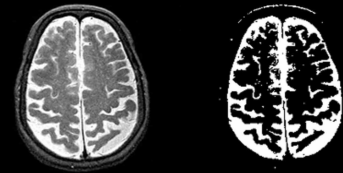


10.4018/978-1-5225-0140-4.ch004

TRADITIONAL COMPUTER VISION

Core Techniques

- ▶ **Thresholding**
Pixels $\geq$ a value $\rightarrow$ set to maximum
- ▶ **Edge Detection**
Detect changes in brightness
- ▶ **Segmentation**
Group regions enclosed by edges
- ▶ **Curve Detection**
Link adjacent edges into shapes
- ▶ **Optical Flow**
Estimate motion between frames

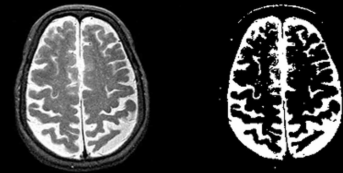


10.4018/978-1-5225-0140-4.ch004

TRADITIONAL COMPUTER VISION

Core Techniques

- ▶ **Thresholding**
Pixels $\geq$ a value $\rightarrow$ set to maximum
- ▶ **Edge Detection**
Detect changes in brightness
- ▶ **Segmentation**
Group regions enclosed by edges
- ▶ **Curve Detection**
Link adjacent edges into shapes
- ▶ **Optical Flow**
Estimate motion between frames

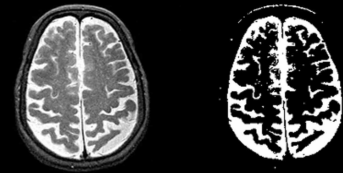


10.4018/978-1-5225-0140-4.ch004

TRADITIONAL COMPUTER VISION

Core Techniques

- ▶ Thresholding
Pixels $\geq$ a value $\rightarrow$ set to maximum
- ▶ Edge Detection
Detect changes in brightness
- ▶ Segmentation
Group regions enclosed by edges
- ▶ Curve Detection
Link adjacent edges into shapes
- ▶ Optical Flow
Estimate motion between frames



10.4018/978-1-5225-0140-4.ch004

Classical pipeline: Pixels $\rightarrow$ Edges $\rightarrow$ Shapes $\rightarrow$ Motion

FEATURE ENGINEERING: SOBEL FILTERS

Edge: a location where image intensity changes abruptly.

Goal: find pixels where the gradient magnitude is large.

Intuition

- ▶ Flat region → small intensity change
- ▶ Object boundary → sharp intensity change

Practical Significance

- ▶ Extract object boundaries
- ▶ Reduce data complexity

Treat the image as a function $f(x, y)$:

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

Numerical Method

Edge strength (gradient magnitude):

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

FEATURE ENGINEERING: SOBEL FILTERS

Edge: a location where image intensity changes abruptly.

Goal: find pixels where the gradient magnitude is large.

Intuition

- ▶ Flat region → small intensity change
- ▶ Object boundary → sharp intensity change

Practical Significance

- ▶ Extract object boundaries
- ▶ Reduce data complexity

Treat the **image as a function** $f(x, y)$:

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

Numerical Method

Edge strength (gradient magnitude):

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

FEATURE ENGINEERING: SOBEL FILTERS

Edge: a location where image intensity changes abruptly.

Goal: find pixels where the gradient magnitude is large.

Intuition

- ▶ Flat region → small intensity change
- ▶ Object boundary → sharp intensity change

Treat the image as a function $f(x, y)$:

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

Edge strength (gradient magnitude):

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

Practical Significance

- ▶ Extract object boundaries
- ▶ Reduce data complexity

Numerical Method

FEATURE ENGINEERING: SOBEL FILTERS

Edge: a location where image intensity changes abruptly.

Goal: find pixels where the gradient magnitude is large.

Intuition

- ▶ Flat region → small intensity change
- ▶ Object boundary → sharp intensity change

Treat the image as a function $f(x, y)$:

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

Edge strength (gradient magnitude):

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

Practical Significance

- ▶ Extract object boundaries
- ▶ Reduce data complexity

Numerical Method

FEATURE ENGINEERING: SOBEL FILTERS

Edge: a location where image intensity changes abruptly.

Goal: find pixels where the gradient magnitude is large.

Intuition

- ▶ Flat region → small intensity change
- ▶ Object boundary → sharp intensity change

Treat the image as a function $f(x, y)$:

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

Edge strength (gradient magnitude):

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

Practical Significance

- ▶ Extract object boundaries
- ▶ Reduce data complexity

Numerical Method

FEATURE ENGINEERING: SOBEL FILTERS

Edge: a location where image intensity changes abruptly.

Goal: find pixels where the gradient magnitude is large.

Intuition

- ▶ Flat region → small intensity change
- ▶ Object boundary → sharp intensity change

Treat the image as a function $f(x, y)$:

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

Edge strength (gradient magnitude):

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

Practical Significance

- ▶ Extract object boundaries
- ▶ Reduce data complexity

Numerical Method

⇒ **Sobel filter**

SOBEL FILTER: VERTICAL EDGE DETECTION

Image Patch (I)		
10	10	200
10	10	200
10	10	200

Dark → Bright

*

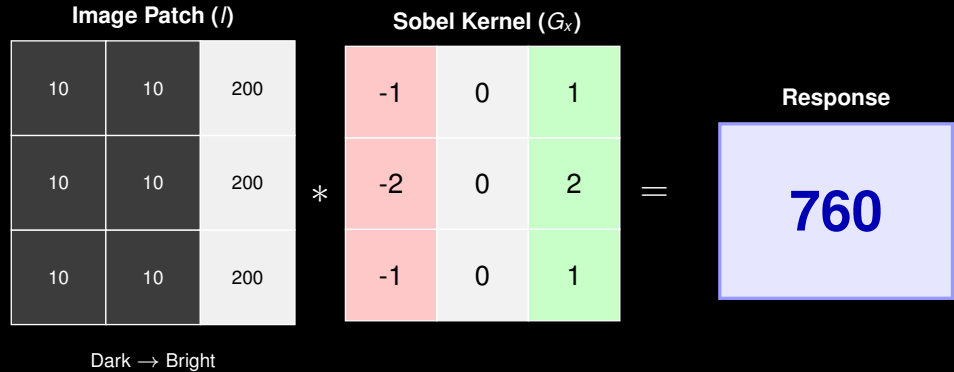
Sobel Kernel (G_x)		
-1	0	1
-2	0	2
-1	0	1

=

Response

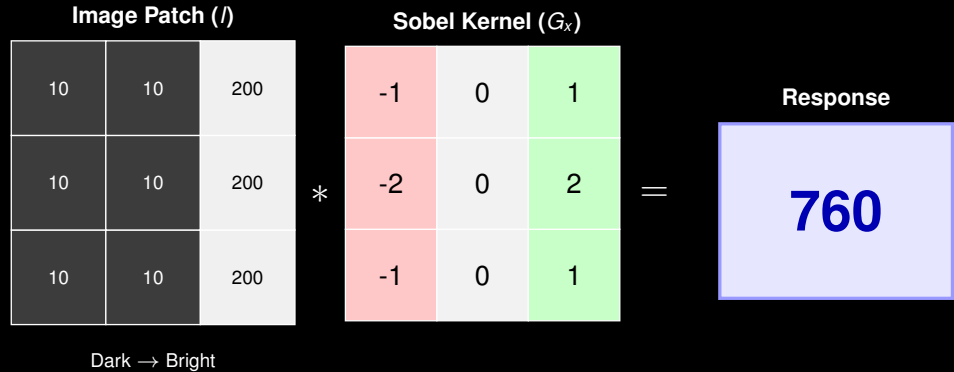
760

SOBEL FILTER: VERTICAL EDGE DETECTION



$$\text{Sum} = (-1 \cdot 10) + (0 \cdot 10) + (1 \cdot 200) + \dots = 760$$

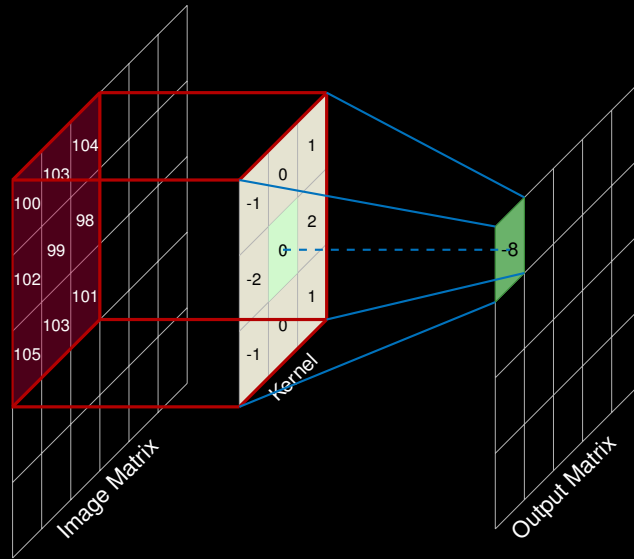
SOBEL FILTER: VERTICAL EDGE DETECTION



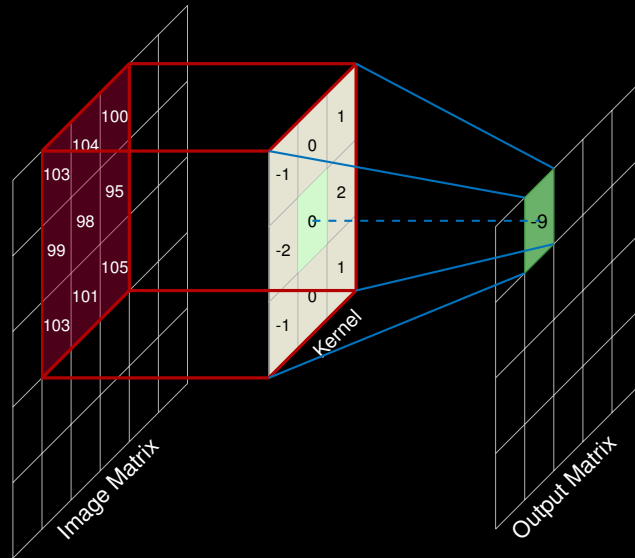
$$\text{Sum} = (-1 \cdot 10) + (0 \cdot 10) + (1 \cdot 200) + \dots = 760$$

The **negative weights** suppress dark pixels, while **positive weights** amplify bright pixels. A high response (760) indicates a strong **vertical edge**.

SOBEL FILTER WITH CONVOLUTION



SOBEL FILTER WITH CONVOLUTION (SLIDE RIGHT + 1)



FEATURE ENGINEERING: TEMPLATE MATCHING

Where's Waldo?



Detected Template



Template

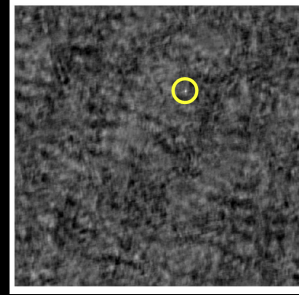
Source: GaTech computer Vision

FEATURE ENGINEERING: TEMPLATE MATCHING

Where's Waldo?



Detected template



Correlation map

Source: GaTech Computer Vision

TEMPLATE MATCHING - CROSS CORRELATION

